

Emacs setup for LaTeX

Moinul Islam

2026-05-31

Table of contents

Preface	3
1 structure	4
2 init.el	5
3 init-ui.el	7
4 init-editing.el	9
5 init-org.el	11
6 init-roam.el	12
7 init-latex.el	16
8 init-company.el	18
9 init-macos.el	19
10 latexmkrc	21
11 MacOS simple setup	22
12 LinuxOS simple setup	29
References	36

Preface

Using the Emacs platform and org-mode is an excellent means of taking daily notes and schedule.

Org-roam is a tool within Org-mode. Org-roam allows for effortless non-hierarchical note-taking: with Org-roam, notes flow naturally, making note-taking fun and easy. Org-roam augments the Org-mode syntax, and will work for anyone already using Org-mode for their personal wiki.

Org-roam leverages the mature ecosystem around Org-mode. For example, it has first-class support for org-ref for citation management, and is able to piggyback off Org's excellent LaTeX and source-block evaluation capabilities.

Org-roam provides these benefits over other tooling:

- **Privacy and Security:** Your personal wiki belongs only to you, entirely offline and in your control. Encrypt your notes with GPG.
- **Longevity of Plain Text:** Unlike web solutions like Roam Research, the notes are first and foremost plain Org-mode files – Org-roam simply builds an auxiliary database to give the personal wiki superpowers. Having your notes in plain-text is crucial for the longevity of your wiki. Never have to worry about proprietary web solutions being taken down. The notes are still functional even if Org-roam ceases to exist.
- **Free and Open Source:** Org-roam is free and open-source, which means that if you feel unhappy with any part of Org-roam, you may choose to extend Org-roam, or open a pull request.
- **Leverage the Org-mode ecosystem:** Over the decades, Emacs and Org-mode has developed into a mature system for plain-text organization. Building upon Org-mode already puts Org-roam light-years ahead of many other solutions.
- **Built on Emacs:** Emacs is also a fantastic interface for editing text, and Org-roam inherits many of the powerful text-navigation and editing packages available to Emacs.

1 structure

This Emacs configuration is well-structured for writing (especially in LaTeX and Org-mode), and personal knowledge management (via Org-roam).

I use this directly as `~/.emacs.d/init.el` file.

```
~/.emacs.d/  
  init.el                ;; Main entry point (already done)  
  custom.el              ;; User customizations  
  backups/               ;; Auto-created  
  auto-saves/            ;; Auto-created  
  lisp/  
    init-ui.el           ;; Fonts, theme, visuals  
    init-editing.el      ;; Smartparens, smex, wraps, spelling  
    init-org.el          ;; Org-mode settings  
    init-roam.el         ;; Org-roam config  
    init-latex.el        ;; AUCTeX + latexmk setup  
    init-company.el      ;; Company-mode auto-completion  
    init-macos.el        ;; macOS-specific setup
```

- Place each file in `~/.emacs.d/lisp/`
- Place `init.el` in `~/.emacs.d/`
- Place `.latexmkrc` in `~/.latexmkrc`

2 init.el

- Keep init.el in ~/.emacs.d/

```
;;; init.el --- Emacs Configuration Entry Point

;; Load path for modular config files
(add-to-list 'load-path (expand-file-name "lisp" user-emacs-directory))

;; Load modules
(require 'init-ui)
(require 'init-editing)
(require 'init-org)
(require 'init-roam)
(require 'init-latex)
(require 'init-company)

;; macOS-specific setup
(when (eq system-type 'darwin)
  (require 'init-macos))

;; -----
;;   Package Manager Setup (package.el + use-package)
;; -----

(require 'package)
(setq package-archives
      '(("melpa" . "https://melpa.org/packages/")
        ("melpa-stable" . "https://stable.melpa.org/packages/")
        ("gnu" . "https://elpa.gnu.org/packages/")
        ("nongnu" . "https://elpa.nongnu.org/nongnu/")))

;; (package-initialize)

(unless package-archive-contents
  (package-refresh-contents))
```

```
;; -----  
;; Save customization to separate file  
;; -----  
(setq custom-file (expand-file-name "custom.el" user-emacs-directory))  
(load custom-file 'noerror)  
  
;;; init.el ends here
```

3 init-ui.el

- Place the init-ui.el file in ~/.emacs.d/lisp/

```
;;; init-ui.el --- Visual appearance and fonts

(use-package doom-themes
  :ensure t
  :config
  (condition-case nil
    (load-theme 'doom-one t)
    (error
     (progn
      (message "doom-one not found, falling back to wombat theme.")
      (load-theme 'wombat t)))))

(defun my/setup-font-based-on-resolution ()
  "Adjust font size automatically based on display resolution, with terminal fallback."
  (interactive)
  (if (display-graphic-p)
      (let* ((display-width (display-pixel-width))
             (font-profiles '((3000 . (:font "Monaco" :size 200))
                              (2500 . (:font "Monaco" :size 190))
                              (1920 . (:font "Monaco" :size 180))
                              (0      . (:font "Monaco" :size 170))))
             (selected-profile
              (cl-loop for (min-width . settings) in font-profiles
                        when (> display-width min-width)
                        return settings)))
          (when selected-profile
            (let ((font-name (plist-get selected-profile :font))
                  (font-size (plist-get selected-profile :size)))
              (when (member font-name (font-family-list))
                (set-face-attribute 'default nil :font font-name :height font-size)
                (message "GUI: Font set to %s (%d)" font-name font-size))))))
      ;; fallback for terminal mode
      (set-face-attribute 'default nil :height 130))
```

```
(message "TTY: Default font height set to 130"))))

(add-hook 'window-setup-hook #'my/setup-font-based-on-resolution)

(global-display-line-numbers-mode 1)
(global-visual-line-mode 1)

(provide 'init-ui)
;;; init-ui.el ends here
```


4 init-editing.el

- Place the init-editing.el file in ~/.emacs.d/lisp/

```
;;; init-editing.el --- General editing enhancements

(use-package smex
  :bind ("M-x" . smex))

(use-package smartparens
  :config
  (smartparens-global-mode t)
  (show-smartparens-global-mode t)
  (sp-pair "\\[" "\\]"))

(use-package rainbow-delimiters
  :hook (prog-mode . rainbow-delimiters-mode))

(use-package rainbow-identifiers
  :hook ((prog-mode latex-mode LaTeX-mode) . rainbow-identifiers-mode))

(use-package adaptive-wrap
  :hook (visual-line-mode . adaptive-wrap-prefix-mode)
  :config
  (setq-default adaptive-wrap-extra-indent 0))

(use-package visual-fill-column
  :config
  (setq-default fill-column 99999))

;; Spell checking
(setq ispell-program-name "aspell")
(setq ispell-dictionary "en")

(dolist (hook '(text-mode-hook org-mode-hook))
  (add-hook hook 'flyspell-mode))
(add-hook 'prog-mode-hook 'flyspell-prog-mode)
```

```

(global-set-key (kbd "C-;") 'flyspell-correct-wrapper)

(use-package flyspell
  :ensure t
  :hook ((text-mode . flyspell-mode)
         (org-mode . flyspell-mode)
         (prog-mode . flyspell-prog-mode))
  :config
  (setq ispell-program-name "aspell"
        ispell-dictionary "en"))

(use-package flyspell-correct
  :ensure t
  :after flyspell
  :bind (:map flyspell-mode-map
             ("C-;" . flyspell-correct-wrapper)))

;; Custom handy shortcut
(setq debug-on-error t)
(electric-indent-mode -1)

(global-set-key (kbd "C-x w")
  (lambda ()
    (interactive)
    (save-excursion
      (forward-char)
      (backward-sexp)
      (let ((pos (point)))
        (forward-sexp)
        (kill-ring-save pos (point))))))

(provide 'init-editing)
;;; init-editing.el ends here

```

5 init-org.el

- Place the init-org.el file in ~/.emacs.d/lisp/

```
;;; init-org.el --- Org mode configuration

(use-package org
  :ensure t
  :hook ((org-mode . org-indent-mode)
        (org-mode . visual-line-mode))
  :config
  (setq org-log-done 'time
        org-startup-folded t
        org-icalendar-include-todo t
        org-hide-emphasis-markers t
        org-pretty-entities t
        org-return-follows-link t))

;; Optional: modern org appearance
(use-package org-modern
  :ensure t
  :hook (org-mode . org-modern-mode))

(provide 'init-org)
;;; init-org.el ends here
```

6 init-roam.el

- Place the init-roam.el file in ~/.emacs.d/lisp/

```
;;; init-roam.el --- Org-roam setup

(use-package org-roam
  :ensure t
  :demand t ;; Ensure org-roam is loaded by default
  :init
  (setq org-roam-v2-ack t)
  :custom
  (org-roam-directory "~/MEGA/org/roam")
  (org-roam-completion-everywhere t)
  :bind (("C-c n l" . org-roam-buffer-toggle)
        ("C-c n f" . org-roam-node-find)
        ("C-c n i" . org-roam-node-insert)
        ("C-c n I" . org-roam-node-insert-immediate)
        ("C-c n p" . my/org-roam-find-project)
        ("C-c n t" . my/org-roam-capture-task)
        ("C-c n b" . my/org-roam-capture-inbox)
        :map org-mode-map
        ("C-M-i" . completion-at-point)
        :map org-roam-dailies-map
        ("Y" . org-roam-dailies-capture-yesterday)
        ("T" . org-roam-dailies-capture-tomorrow))
  :bind-keymap
  ("C-c n d" . org-roam-dailies-map)
  :config
  (require 'org-roam-dailies) ;; Ensure the keymap is available
  (org-roam-db-autosync-mode))

(defun org-roam-node-insert-immediate (arg &rest args)
  (interactive "P")
  (let ((args (push arg args))
        (org-roam-capture-templates (list (append (car org-roam-capture-templates)
                                                    '(:immediate-finish t))))))
```

```

    (apply #'org-roam-node-insert args)))

(defun my/org-roam-filter-by-tag (tag-name)
  (lambda (node)
    (member tag-name (org-roam-node-tags node))))

(defun my/org-roam-list-notes-by-tag (tag-name)
  (mapcar #'org-roam-node-file
    (seq-filter
      (my/org-roam-filter-by-tag tag-name)
      (org-roam-node-list))))

(defun my/org-roam-refresh-agenda-list ()
  (interactive)
  (setq org-agenda-files (my/org-roam-list-notes-by-tag "Project")))

;; Build the agenda list the first time for the session
(my/org-roam-refresh-agenda-list)

(defun my/org-roam-project-finalize-hook ()
  "Adds the captured project file to `org-agenda-files' if the
capture was not aborted."
  ;; Remove the hook since it was added temporarily
  (remove-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Add project file to the agenda list if the capture was confirmed
  (unless org-note-abort
    (with-current-buffer (org-capture-get :buffer)
      (add-to-list 'org-agenda-files (buffer-file-name)))))

(defun my/org-roam-find-project ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Select a project file to open, creating it if necessary
  (org-roam-node-find
    nil
    nil
    (my/org-roam-filter-by-tag "Project")
    :templates
    '(("p" "project" plain "* Goals\n\n%?\n\n* Tasks\n\n** TODO Add initial tasks\n\n* Dates\n\n"))))

```

```

      :if-new (file+head "%<%Y%m%d%H%M%S>-${slug}.org" "#+title: ${title}\n#+category: ${tit
      :unnarrowed t))))

(defun my/org-roam-capture-inbox ()
  (interactive)
  (org-roam-capture- :node (org-roam-node-create)
                     :templates '(("i" "inbox" plain "* %?"
                                   :if-new (file+head "Inbox.org" "#+title: Inbox\n")))))

(defun my/org-roam-capture-task ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Capture the new task, creating the project file if necessary
  (org-roam-capture- :node (org-roam-node-read
                           nil
                           (my/org-roam-filter-by-tag "Project"))
                    :templates '(("p" "project" plain "** TODO %?"
                                   :if-new (file+head+olp "%<%Y%m%d%H%M%S>-${slug}.org"
                                                           "#+title: ${title}\n#+category: ${
                                                           ("Tasks"))))))))

(defun my/org-roam-copy-todo-to-today ()
  (interactive)
  (let ((org-refile-keep t) ;; Set this to nil to delete the original!
        (org-roam-dailies-capture-templates
         '(("t" "tasks" entry "%?"
             :if-new (file+head+olp "%<%Y-%m-%d>.org" "#+title: %<%Y-%m-%d>\n" ("Tasks")))))
        (org-after-refile-insert-hook #'save-buffer)
        today-file
        pos)
    (save-window-excursion
      (org-roam-dailies--capture (current-time) t)
      (setq today-file (buffer-file-name))
      (setq pos (point)))

    ;; Only refile if the target file is different than the current file
    (unless (equal (file-truename today-file)
                   (file-truename (buffer-file-name)))
      (org-refile nil nil (list "Tasks" today-file nil pos))))

```

```
(add-to-list 'org-after-todo-state-change-hook
  (lambda ()
    (when (equal org-state "DONE")
      (my/org-roam-copy-todo-to-today))))

(provide 'init-roam)
;;; init-roam.el ends here
```

7 init-latex.el

- Place the init-latex.el file in ~/.emacs.d/lisp/

```
;;; init-latex.el --- LaTeX / AUCTeX + latexmk configuration

(use-package tex
  :ensure auctex
  :defer t
  :hook (LaTeX-mode . my/latex-setup)
  :config
  (setq TeX-auto-save t
        TeX-parse-self t
        TeX-save-query nil
        TeX-PDF-mode t
        TeX-source-correlate-mode t
        TeX-source-correlate-method 'synctex)

  (setq TeX-view-program-selection
        '(((output-pdf "PDF Viewer"))))

  ;; Use Skim with SyncTeX or fallback to Preview.app
  (setq TeX-view-program-list
        '(("PDF Viewer" "/Applications/Skim.app/Contents/SharedSupport/displayline -b -g %n %s")
          ("PDF Viewer" "/Applications/Preview.app/Contents/MacOS/Preview -f %n %s")))

  (add-to-list 'TeX-command-list
    '("LatexMk"
      "latexmk -pdf -synctex=1 -interaction=nonstopmode %s"
      TeX-run-Tex nil t
      :help "Run latexmk for PDF output"))

  (setq TeX-command-default "LatexMk"))

(defun my/latex-setup ()
  "My custom LaTeX setup."
  (turn-on-reftex)
  (setq reftex-plug-into-AUCTeX t))
```



```
(flyspell-mode 1)
(LaTeX-math-mode 1)
(visual-line-mode 1))

(provide 'init-latex)
;;; init-latex.el ends here
```

8 init-company.el

- Place the init-company.el file in ~/.emacs.d/lisp/

```
;;; init-company.el --- Company-mode configuration

(use-package company
  :hook ((after-init . global-company-mode)
        (LaTeX-mode . company-mode))
  :config
  (setq company-idle-delay 0.2
        company-minimum-prefix-length 2
        company-tooltip-align-annotations t
        company-dabbrev-downcase nil))

(provide 'init-company)
;;; init-company.el ends here
```

9 init-macos.el

- Place the init-macos.el file in ~/.emacs.d/lisp/

```
;; -----  
;;   MacOS-Specific Setup for Emacs  
;; -----  
  
;;   Startup: Performance tuning  
(setq native-comp-async-report-warnings-errors nil) ; silence native-comp warnings  
(setq gc-cons-threshold (* 50 1000 1000))           ; increase GC threshold for faster start  
  
(add-hook 'emacs-startup-hook  
  (lambda ()  
    (message " Emacs ready in %.2f seconds with %d GCs."  
             (float-time (time-subtract after-init-time before-init-time))  
             gcs-done)))  
  
;;   Environment: Mac shell paths  
(use-package exec-path-from-shell  
  :if (memq window-system '(mac ns x))  
  :config  
  (exec-path-from-shell-initialize))  
  
;; Add MacPorts paths for TeXLive and Hunspell  
(let ((my-paths '("/opt/local/libexec/texlive/texbin"  
                  "/opt/local/bin")))  
  (setenv "PATH" (concat (mapconcat #'identity my-paths ":") ":" (getenv "PATH")))  
  (setq exec-path (append my-paths exec-path)))  
  
;;   Visuals: Theme, font, retina, fullscreen  
(when (eq system-type 'darwin)  
  ;; Prefer dark mode and unified title bar  
  (add-to-list 'default-frame-alist '(ns-appearance . dark))  
  (add-to-list 'default-frame-alist '(ns-transparent-titlebar . t)))  
  
(setq frame-resize-pixelwise t) ; Retina/HiDPI fix
```

```

(add-to-list 'default-frame-alist '(fullscreen . maximized)) ; Start maximized
(set-fontset-font t 'symbol (font-spec :family "Apple Color Emoji") nil 'prepend)

;; Mouse & Trackpad: Smooth scrolling
(setq mouse-wheel-scroll-amount '(1 ((shift) . 1))) ; 1 line at a time
(setq mouse-wheel-progressive-speed nil)
(setq scroll-step 1)

;; Clipboard and Input
(setq select-enable-clipboard t)
(setq save-interprogram-paste-before-kill t)
(setq use-dialog-box nil) ; no popups

;; Optional: Better Japanese keyboard handling (¥ = \)
;; (define-key global-map [?¥] [?\])

;; UI Hygiene (uncomment if desired)
;; (menu-bar-mode -1)
;; (tool-bar-mode -1)
;; (scroll-bar-mode -1)

(provide 'init-macos)

```

10 latexmkrc

- Place .latexmkrc in ~/.latexmkrc

```
# Generate pdf using pdflatex (-pdf)
$pdf_mode=1;

# Use bibtex if a .bib file exists
$bibtex_use=1;

# Define command to compile with pdfsync support and nonstopmode
$pdflatex= 'pdflatex -synctex=1 --interaction=nonstopmode -file-line-error';

# Continuous compiling
$preview_continuous_mode=1;
```

11 MacOS simple setup

```
;;; init.el --- MacOSX

;; -----
;;   Package Management
;; -----
(require 'package)

(setq package-enable-at-startup nil)

;; Add multiple reputable package archives
(setq package-archives
  '(("melpa"      . "https://melpa.org/packages/")
    ("melpa-stable" . "https://stable.melpa.org/packages/")
    ("gnu"        . "https://elpa.gnu.org/packages/")
    ("nongnu"     . "https://elpa.nongnu.org/nongnu/")
    ("org"        . "https://orgmode.org/elpa/")))

(package-initialize)

(unless (package-installed-p 'use-package)
  (package-refresh-contents)
  (package-install 'use-package))

(require 'use-package)
(setq use-package-always-ensure t)

;; -----
;;   Org-mode Setup (with macOS bindings)
;; -----
(use-package org
  :ensure t
  :hook ((org-mode . visual-line-mode)
        (org-mode . variable-pitch-mode))
  :bind (("C-c c" . org-capture)           ;; Org capture shortcut
```

```

      :map org-mode-map
      ("s-d" . org-deadline)          ;; + d → set deadline
      ("s-s" . org-schedule)          ;; + s → schedule
      ("s-a" . org-agenda))           ;; + a → agenda

:config
(setq org-directory "~/org/")
(setq org-agenda-files ('("~/org/"))
(setq org-log-done 'time)
(setq org-startup-indented t)
(setq org-hide-emphasis-markers t)
(setq org-startup-with-inline-images t)
(setq org-deadline-warning-days 7)
(setq org-agenda-span 'week)
(setq org-read-date-popup-calendar t)
(setq calendar-week-start-day 1))

;; -----
;;   Org-roam Setup
;; -----
(use-package org-roam
  :ensure t
  :demand t ;; Ensure org-roam is loaded by default
  :init
  (setq org-roam-v2-ack t)
  :custom
  (org-roam-directory "~/MEGA/org/roam")
  (org-roam-completion-everywhere t)
  :bind (("C-c n l" . org-roam-buffer-toggle)
        ("C-c n f" . org-roam-node-find)
        ("C-c n i" . org-roam-node-insert)
        ("C-c n I" . org-roam-node-insert-immediate)
        ("C-c n p" . my/org-roam-find-project)
        ("C-c n t" . my/org-roam-capture-task)
        ("C-c n b" . my/org-roam-capture-inbox)
        :map org-mode-map
        ("C-M-i" . completion-at-point)
        :map org-roam-dailies-map
        ("Y" . org-roam-dailies-capture-yesterday)
        ("T" . org-roam-dailies-capture-tomorrow))
  :bind-keymap
  ("C-c n d" . org-roam-dailies-map)
:config

```

```

(require 'org-roam-dailies) ;; Ensure the keymap is available
(org-roam-db-autosync-mode))

(defun org-roam-node-insert-immediate (arg &rest args)
  (interactive "P")
  (let ((args (push arg args))
        (org-roam-capture-templates (list (append (car org-roam-capture-templates)
                                                    '(:immediate-finish t))))))
    (apply #'org-roam-node-insert args)))

(defun my/org-roam-filter-by-tag (tag-name)
  (lambda (node)
    (member tag-name (org-roam-node-tags node))))

(defun my/org-roam-list-notes-by-tag (tag-name)
  (mapcar #'org-roam-node-file
          (seq-filter
           (my/org-roam-filter-by-tag tag-name)
           (org-roam-node-list))))

(defun my/org-roam-refresh-agenda-list ()
  (interactive)
  (setq org-agenda-files (my/org-roam-list-notes-by-tag "Project")))

;; Build the agenda list the first time for the session
(my/org-roam-refresh-agenda-list)

(defun my/org-roam-project-finalize-hook ()
  "Adds the captured project file to `org-agenda-files' if the
capture was not aborted."
  ;; Remove the hook since it was added temporarily
  (remove-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Add project file to the agenda list if the capture was confirmed
  (unless org-note-abort
    (with-current-buffer (org-capture-get :buffer)
      (add-to-list 'org-agenda-files (buffer-file-name)))))

(defun my/org-roam-find-project ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook))

```



```

;; Select a project file to open, creating it if necessary
(org-roam-node-find
 nil
 nil
 (my/org-roam-filter-by-tag "Project"))
:templates
'(("p" "project" plain "* Goals\n\n%\n\n* Tasks\n\n** TODO Add initial tasks\n\n* Dates\n\n"
  :if-new (file+head "%<%Y%m%d%H%M%S>-${slug}.org" "#+title: ${title}\n#+category: ${title}"
    :unnarrowed t))))

(defun my/org-roam-capture-inbox ()
  (interactive)
  (org-roam-capture- :node (org-roam-node-create)
    :templates '(("i" "inbox" plain "* %?"
      :if-new (file+head "Inbox.org" "#+title: Inbox\n")))))

(defun my/org-roam-capture-task ()
  (interactive)
  ;; Add the project file to the agenda after capture is finished
  (add-hook 'org-capture-after-finalize-hook #'my/org-roam-project-finalize-hook)

  ;; Capture the new task, creating the project file if necessary
  (org-roam-capture- :node (org-roam-node-read
    nil
    (my/org-roam-filter-by-tag "Project"))
    :templates '(("p" "project" plain "** TODO %?"
      :if-new (file+head+olp "%<%Y%m%d%H%M%S>-${slug}.org"
        "#+title: ${title}\n#+category: ${title}"
        ("Tasks")))))

(defun my/org-roam-copy-todo-to-today ()
  (interactive)
  (let ((org-refile-keep t) ;; Set this to nil to delete the original!
    (org-roam-dailies-capture-templates
      '(("t" "tasks" entry "%?"
        :if-new (file+head+olp "%<%Y-%m-%d>.org" "#+title: %<%Y-%m-%d>\n" ("Tasks")))))
    (org-after-refile-insert-hook #'save-buffer)
    today-file
    pos)
    (save-window-excursion
      (org-roam-dailies--capture (current-time) t)
      (setq today-file (buffer-file-name))

```

```

    (setq pos (point)))

;; Only refile if the target file is different than the current file
(unless (equal (file-truename today-file)
              (file-truename (buffer-file-name)))
  (org-refile nil nil (list "Tasks" today-file nil pos))))

(add-to-list 'org-after-todo-state-change-hook
  (lambda ()
    (when (equal org-state "DONE")
      (my/org-roam-copy-todo-to-today))))

;; -----
;;   Visual Enhancements
;; -----
(use-package org-bullets
  :hook (org-mode . org-bullets-mode))

(use-package visual-fill-column
  :hook (org-mode . visual-fill-column-mode)
  :config
  (setq visual-fill-column-width 100
        visual-fill-column-center-text t))

(use-package solarized-theme
  :config
  (load-theme 'solarized-dark t))

(global-display-line-numbers-mode 1)
(add-hook 'org-mode-hook (lambda () (display-line-numbers-mode 0)))

;; Optional: Nicer Org UI
(use-package org-modern
  :config (global-org-modern-mode))

(use-package org-appear
  :hook (org-mode . org-appear-mode))

;; -----
;;   macOS Enhancements and Font
;; -----
(when (eq system-type 'darwin)

```

```

;; Set font size to 18pt and background to black
(add-to-list 'default-frame-alist '(font . "Menlo-18"))
(add-to-list 'default-frame-alist '(background-color . "black"))
(add-to-list 'default-frame-alist '(foreground-color . "white"))
(setq mac-option-modifier 'meta)
(setq mac-command-modifier 'super)
(setq mac-right-option-modifier nil))

;; -----
;;  Spell Checking & Dictionary
;; -----
(setq ispell-program-name "aspell")
(setq ispell-dictionary "en")      ;; default dictionary

;; Enable Flyspell in text and org modes
(dolist (hook '(text-mode-hook org-mode-hook))
  (add-hook hook 'flyspell-mode))

;; Optional: enable in comments for code
(add-hook 'prog-mode-hook 'flyspell-prog-mode)

;; Set correction keybinding (use M-$ to check word)
(global-set-key (kbd "C-;") 'flyspell-correct-wrapper)

(use-package flyspell
  :ensure t
  :hook ((text-mode . flyspell-mode)
        (org-mode . flyspell-mode)
        (prog-mode . flyspell-prog-mode))
  :config
  (setq ispell-program-name "aspell"
        ispell-dictionary "en"))

(provide 'init)
;;; init.el ends here
(custom-set-variables
 ;; custom-set-variables was added by Custom.
 ;; If you edit it by hand, you could mess it up, so be careful.
 ;; Your init file should contain only one such instance.
 ;; If there is more than one, they won't work right.
 '(package-selected-packages nil))
(custom-set-faces

```

```
;; custom-set-faces was added by Custom.  
;; If you edit it by hand, you could mess it up, so be careful.  
;; Your init file should contain only one such instance.  
;; If there is more than one, they won't work right.  
)
```

12 LinuxOS simple setup

```
;;; init.el --- Ubuntu

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Startup optimization
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(setq gc-cons-threshold (* 100 1024 1024))

(add-hook 'emacs-startup-hook
  (lambda ()
    (setq gc-cons-threshold (* 20 1024 1024))))

(setq native-comp-async-report-warnings-errors nil)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Package system
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(require 'package)

(setq package-archives
  '(("gnu"      . "https://elpa.gnu.org/packages/")
    ("nongnu"   . "https://elpa.nongnu.org/nongnu/")
    ("melpa"    . "https://melpa.org/packages/")))

(package-initialize)

(unless package-archive-contents
  (package-refresh-contents))

(unless (package-installed-p 'use-package)
  (package-install 'use-package))

(require 'use-package)
```

```

(setq use-package-always-ensure t)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; UI / Appearance
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package solarized-theme
  :config
  (load-theme 'solarized-dark t))

(set-face-attribute 'default nil
                   :font "DejaVu Sans Mono"
                   :height 140)

(setq inhibit-startup-screen t
      ring-bell-function 'ignore)

;; Ubuntu-friendly UI
(menu-bar-mode 1)
(tool-bar-mode -1)
(scroll-bar-mode 1)

(global-hl-line-mode 1)
(column-number-mode 1)
(display-time-mode 1)
(global-visual-line-mode 1)

(add-hook 'prog-mode-hook #'display-line-numbers-mode)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Which-key
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package which-key
  :config
  (which-key-mode))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Completion system (modern)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package vertico

```

```

:init (vertico-mode))

(use-package orderless
  :custom
  (completion-styles '(orderless basic)))

(use-package marginalia
  :init (marginalia-mode))

(use-package consult)

(use-package embark
  :bind (("C-." . embark-act)))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Corfu (completion UI)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package corfu
  :init (global-corfu-mode)
  :custom
  (corfu-auto t)
  (corfu-auto-delay 0.2)
  (corfu-auto-prefix 2))

(use-package cape
  :init
  (add-to-list 'completion-at-point-functions #'cape-file))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Smart editing
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package smartparens
  :hook ((prog-mode LaTeX-mode org-mode) . smartparens-mode)
  :config (require 'smartparens-config))

(use-package rainbow-delimiters
  :hook (prog-mode . rainbow-delimiters-mode))

(use-package highlight-parentheses
  :config (global-highlight-parentheses-mode))

```

```

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Spell checking
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(setq ispell-program-name "hunspell"
      ispell-local-dictionary "en_US")

(add-hook 'text-mode-hook #'flyspell-mode)
(add-hook 'prog-mode-hook #'flyspell-prog-mode)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; PDF Tools (interactive PDF)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package pdf-tools
  :config
  (pdf-tools-install)
  (setq-default pdf-view-display-size 'fit-width))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Org mode
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package org
  :hook (org-mode . visual-line-mode)
  :config
  (setq org-log-done 'time
        org-startup-indented t
        org-hide-emphasis-markers t))

(global-set-key (kbd "C-c a") #'org-agenda)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Org-roam
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package org-roam
  :custom
  (org-roam-directory
   (file-truename "~/MEGA/org/roam"))
  :bind (("C-c n l" . org-roam-buffer-toggle)
        ("C-c n f" . org-roam-node-find))

```



```

("C-c n i" . org-roam-node-insert))
:config
(org-roam-db-autosync-mode))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Zotero / citations (Citar)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package citar
  :custom
  (citar-bibliography
   '("~/MEGA/references/references.bib"))
  :bind (("C-c b" . citar-insert-citation)))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; ESS (R)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package ess
  :mode ("\\.R\\'" . R-mode)
  :config
  (setq ess-indent-offset 2))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; LaTeX + latexmk + SyncTeX (INTERACTIVE PDF CORE)
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package tex
  :ensure auctex
  :hook
  ((LaTeX-mode . visual-line-mode)
   (LaTeX-mode . flyspell-mode)
   (LaTeX-mode . LaTeX-math-mode)
   (LaTeX-mode . TeX-source-correlate-mode))
  :config
  (setq TeX-auto-save t
        TeX-parse-self t
        TeX-save-query nil
        TeX-PDF-mode t

        ;; IMPORTANT: interactive PDF support
        TeX-source-correlate-method 'synctex

```

```

TeX-source-correlate-start-server t

;; latexmk default
TeX-command-default "LatexMk"

;; always use PDF Tools
TeX-view-program-selection '((output-pdf "PDF Tools"))))

(use-package auctex-latexmk
  :after tex
  :config
  (auctex-latexmk-setup))

(use-package reftex
  :hook (LaTeX-mode . turn-on-reftex)
  :config
  (setq reftex-plug-into-AUCTeX t))

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Markdown / Quarto
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(use-package markdown-mode)
(use-package poly-markdown)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Backup system
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(setq backup-directory-alist
  `(("." . "~/ .emacs.d/backups"))
  auto-save-default t)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Keybindings
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

(global-set-key (kbd "<f5>") #'revert-buffer)
(global-set-key (kbd "C-x k") #'kill-current-buffer)

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Finish

```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

```
(message "Academic Emacs ready")
```

References